

CHAPTER

8

Subroutines

One key motivation of subroutines is to enable the reuse of a portion of the program code that carries out a specific task. Moreover, using subroutines can increase the quality and reliability of programs. A subroutine is also called a procedure, a function, or a routine.

One mistake which new software programmers often make is to duplicate code within a program by copy-and-paste. This programming style makes the program less modularized and difficult to read and debug. It increases the cost of developing and maintaining a large software project substantially.

Breaking the program codes into subroutines has two significant advantages.

- First, it decomposes a complex task into several simpler and more manageable subtasks, which makes the design, development, and debugging of each subtask much easier. We can develop and test each subtask separately, making it easier to find errors.
- Second, subtasks can be reused in a program or other subtasks repeatedly without duplicating any code, which saves time and effort by eliminating redundant development work.

*"One of my most
productive days was
throwing away 1000
lines of code."*

Ken Thompson, early
developer of UNIX OS

A subroutine usually takes some input arguments and returns a result, when the subroutine exits. For example, when a subroutine finds the maximum of an array of elements, it may take two arguments: the memory address (a pointer in C language) of the array and the number of items in this array. It may also return the maximum value or the index (or the address of the maximum value). Using arguments makes a subroutine versatile and suitable for a variety of uses.

Implementing a subroutine using a high-level programming language such as Java, C, or C++ differs significantly from using an assembly language. Specifically, when a programmer develops a subroutine in assembly, we must consider two special issues.

1. Preserve and recover the caller's environment. The subroutine should be non-intrusive and avoid destroying the content of relevant registers, which are still meaningful for the caller. A subroutine takes two steps to preserve and to recover the caller's runtime environment.
 - First, the subroutine saves the environment by pushing registers to be used in the subroutine onto the stack at the beginning of the subroutine.
 - Second, the subroutine restores the environment by popping the content off the stack to registers at the end of the subroutine.
2. The standard of application binary interface specifies the protocol of passing input arguments to a subroutine and returning the result back to the caller. If subroutines developed in assembly follow the standard, a C program can call these subroutines. Following the standard also facilitates sharing code between different programmers.

8.1 Calling a Subroutine

An assembly program can use the branch and link (BL) instruction to call a subroutine. The BL instruction performs the following two operations:

1. Configure the link register (LR) as the memory address of the next instruction immediately after the BL instruction ($PC + 4$ in Cortex-M), and
2. Adjust the program counter (PC) as the memory address of the very first instruction of the subroutine.

Register LR stores the memory address of the instruction to be executed immediately after a subroutine exits. The address held in LR is also called *return address*. The return address is the memory address of the instruction immediately after the BL instruction in the caller program. Since a BL instruction takes 32 bits (*i.e.*, 4 bytes) in memory, the memory address of the instruction immediately after the BL instruction is $PC + 4$.

There are two different approaches to return from a subroutine.

- The first one is to run the branch and exchange instruction "BX LR".
- The second one is to pop the LR value off the stack into PC, *i.e.*, "POP {PC}", if the subroutine has already pushed LR onto the stack.

Figure 8-1 shows an example in which the main program calls the subroutine *foo*, and the subroutine *foo* then calls the subroutine *bar*. Because each BL instruction takes 32 bits (i.e., 4 bytes), the address of the next instruction immediately after a BL instruction is PC + 4.

- When the main program uses “BL *foo*” to call the subroutine *foo*, the processor carries out two operations: (1) set LR to PC₁ + 4, and (2) set PC to the memory address of the subroutine *foo*.
- When *foo* returns by executing “BX LR”, the processor copies the content of LR to PC, making the next instruction after “BL *foo*” in the main program to start the execution.

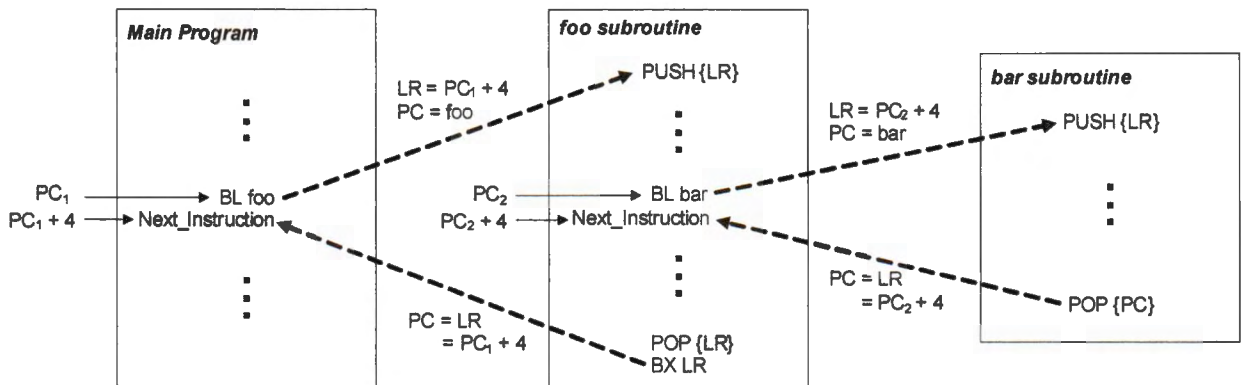


Figure 8-1. An example of calling subroutines. A subroutine should preserve the link register (LR) in the stack if this subroutine calls any subroutines.

When the subroutine *foo* calls another subroutine *bar*, the register *foo* should preserve LR in the stack at the beginning of the subroutine. In the example shown in Figure 8-1,

- When *foo* calls *bar*, “BL *bar*” sets LR to PC₂ + 4.
- When *bar* executes “POP {PC}”, the processor sets LR to PC₂ + 4
- If *foo* did not preserve and recover LR, i.e., *foo* did not have “PUSH {LR}” and “POP {LR}” in the code, the “BX LR” instruction at the end of *foo* would not be able to return the control to the main program correctly. The processor would mistakenly set PC to PC₂ + 4, instead of PC₁ + 4 when *foo* exits.

**Preserve Link Register (LR)
in a subroutine if it calls
another subroutine.**

- Therefore, *foo* should push LR onto the stack if it calls another subroutine or itself.

8.2 Stack

A stack is a last-in-first-out (LIFO) data structure, as shown in Figure 8-2. A stack has two fundamental operations: *push* and *pop*. The push operation adds an item to the top of the stack. The pop operation removes the item that was added last.

A stack also refers to a contiguous region in the data memory that software programs or processors use to hold a stack data structure. The stack pointer (SP) holds the memory address of the top of the stack. A program can utilize stacks to preserve and recovery the runtime environment when it calls a subroutine.

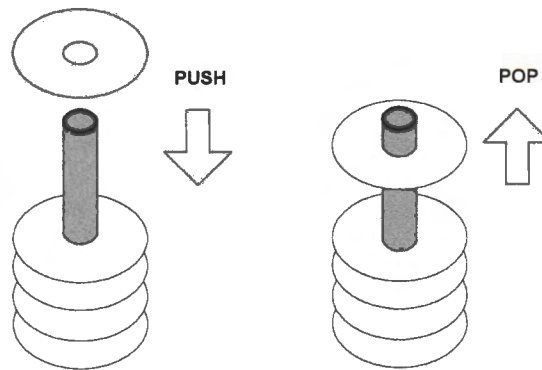


Figure 8-2. A stack is last-in-first-out (LIFO).

Per the direction in which the stack grows, a stack can be either descending or ascending, as shown in Figure 8-3.

- **Descending stack.** When the content of a register is pushed onto the stack, the stack pointer (SP) is decreased by 4 if it is a descending stack. The stack starts at a high memory address and grows downward, *i.e.*, in the direction toward lower memory addresses.
- **Ascending stack.** When a program pushes a register onto the stack, SP is increased by 4. The stack starts at a low memory address grows upward, *i.e.*, in the direction toward high memory addresses.

Also, a stack can be either a *full stack* or an *empty stack*.

- **Full stack.** If the memory location pointed by SP holds the last item that was pushed onto the stack, this stack is called a full stack.
- **Empty stack.** If the memory location pointed by SP is an empty spot and will store the next item to be pushed onto the stack, this stack is referred to as an empty stack.

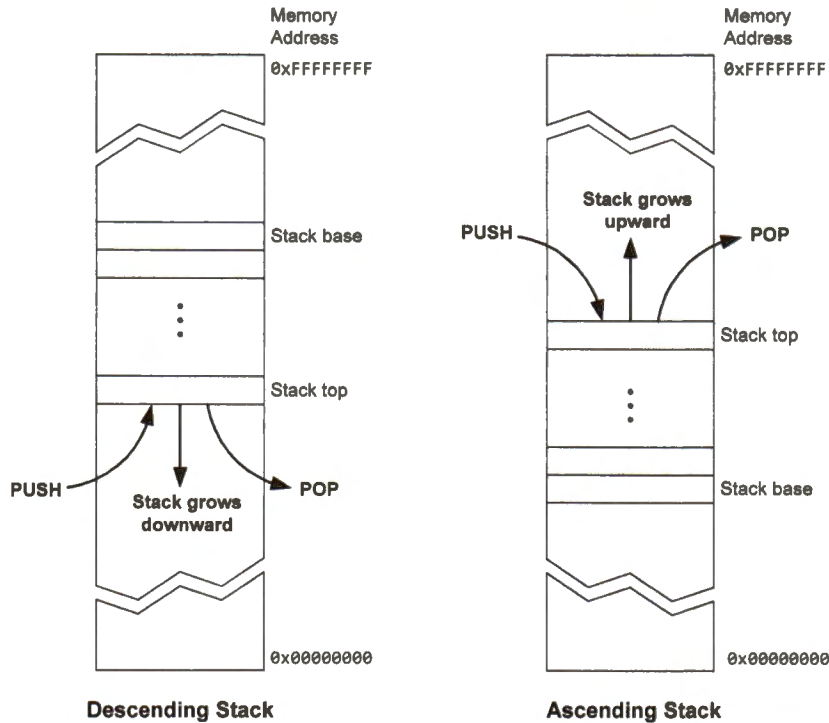


Figure 8-3. Comparing descending stack and ascending stack.

With a combination of the growth direction and the validness of the memory location pointed by the stack pointer, there are four types of stacks, including *full descending stack*, *full ascending stack*, *empty descending stack*, and *empty ascending stack*.

8.3 Implementation of Stack via STM and LDM

Table 8-1 shows the assembly instructions equivalent to the push and pop operations for these four types of stacks. For each push and pop operation, there are two implementations. Chapter 5.5 has explained these STM and LDM instructions.

Stack Name	Push		Pop	
	Equivalent	Alternative	Equivalent	Alternative
Full Descending(FD)	STMFD SP!,list	STMDB SP!,list	LDMFD SP!,list	LDMIA SP!,list
Empty Descending(ED)	STMED SP!,list	STMDA SP!,list	LDMED SP!,list	LDMIB SP!,list
Full Ascending(FA)	STMFA SP!,list	STMIB SP!,list	LDMFA SP!,list	LDMDA SP!,list
Empty Ascending(EA)	STMEA SP!,list	STMIA SP!,list	LDMEA SP!,list	LDMDB SP!,list

Table 8-1. Implementation of PUSH and POP by using STM and LDM instructions

ARM Cortex-M uses *full descending* stacks, as shown in Figure 8-4. For full-descending stacks, the push and pop operations are equivalent to **STMDB** and **LDMIA**, respectively.

Moreover, a Cortex-M processor supports two stacks: one main stack and one process stack. Therefore, there are two stack pointers: MSP and PSP. The stack pointer (SP) is a shadow of MSP or PSP. Chapter 23.1 discusses the usage of these two stacks. The initial value of these two stack pointers can be programmed. Typically, the system stack is placed in the top region of the system memory region.

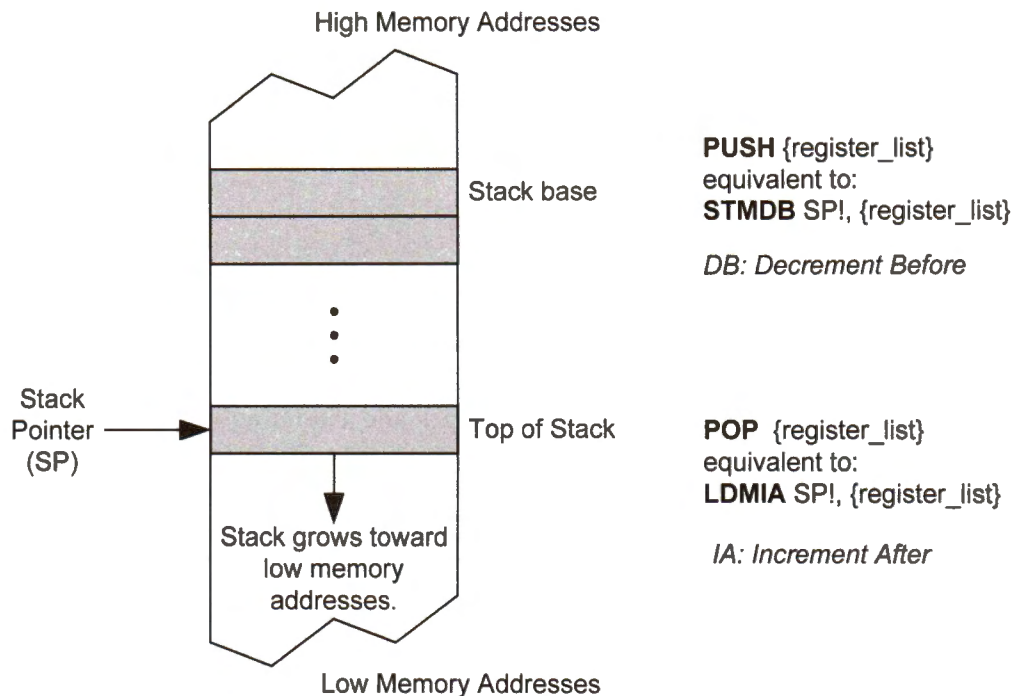


Figure 8-4. Cortex-M uses *full descending* stack

8.4 Preserving Runtime Environment via Stack

If a subroutine uses a register to hold some variable, the subroutine often must preserve the value of this register in the stack when the subroutine starts, and recover the register value from the stack when the subroutine exits. In this process, the callee preserves the caller's runtime environment.

Example 8-1 shows the motivation why a subroutine must preserve the runtime environment. The caller uses register **r4** as a loop variable looping from 0 to 100. The

loop body calls a subroutine named *foo*. However, the subroutine *foo* also uses register *r4* as a local variable. If the subroutine does not preserve the content in register *r4*, *r4* is mistakenly set to 10 after the subroutine completes. Thus, the loop in the caller does not perform as desired.

Caller Program	Subroutine/Callee
<pre> MOV r4, #100 ... BL foo ... ADD r4, r4, #1 ; r4 = 101, not 11 </pre>	<pre> foo PROC ... MOV r4, #10 ; foo changes r4 ... BX LR ENDP </pre>

Example 8-1. The loop in the caller program is not executed as desired.

To solve this issue, the subroutine stores the value of register *r4* onto the stack before executing any other code, and then restores the content to *r4* immediately before the subroutine returns. The stack push and pop operations preserve and recover the runtime environment, as illustrated below.

Caller Program	Subroutine/Callee
<pre> ; main.s MOV r4, #100 ... BL foo ... ADD r4, r4, #1 ; r4 = 101, not 11 </pre>	<pre> foo PROC ... PUSH {r4} ; preserve r4 ... MOV r4, #10 ; foo changes r4 ... POP {r4} ; Recover r4 BX LR ENDP </pre>

Example 8-2. A subroutine preserves register values in the stack at the beginning and recovers register values from the stack before the subroutine exits.

The ARM embedded application binary interface (EABI) requires that a subroutine must preserve the content in registers *r4* - *r11* and *r13* (SP) if this subroutine changes the value of these registers.

However, a subroutine is not required to preserve registers *r0*-*r3*, also called *scratch registers*. Thus, if the caller requires that the value in registers *r0*-*r3* remains unchanged, the caller should push these registers onto the stack before it calls a subroutine.

Table 8-2 summarizes the register usage. Subroutines should meet the following requirements.

- Besides registers *r0*-*r3*, a subroutine also does not preserve the intra-procedure-call register *r12* (IP) and the link register *r14* (LR).

- Register r9 is platform dependent and has different purposes. For example, r9 can be a variable register to hold a local variable.
- Register IP allows a routine and any subroutine it calls to share an intermediate value. When the caller executes a BL instruction, the value of IP may be changed if the memory address of the destination instruction is beyond the range of the BL instruction. Therefore, software should not use register IP as a general-purpose register to hold important values.
- It is not required to preserve register LR. However, if a subroutine A() calls another subroutine B(), the subroutine A() should preserve LR in the stack (see Chapter 8.1).

Register	Usage	Subroutine Preserved	Notes
r0	Argument 1 and return value	No	If the 1 st argument has 64 bits, r1:r0 hold it (r1 is upper word, r0 is bottom word). If the 2 nd argument has 64 bits, r3:r2 hold it. If more than 4 arguments, use the stack. If the return has 64 bits, then r1:r0 hold it. If the return has 128 bits, then r0-r3 hold it.
r1	Argument 2	No	
r2	Argument 3	No	
r3	Argument 4	No	
r4	General-purpose V1	Yes	Variable registers for holding local variables.
r5	General-purpose V2	Yes	
r6	General-purpose V3	Yes	
r7	General-purpose V4	Yes	
r8	General-purpose V5	Yes	
r9	Platform specific/V6	No	Usage is platform-dependent.
r10	General-purpose V7	Yes	Variable registers for holding local variables
r11	General-purpose V8	Yes	
r12 (IP)	Intra-procedure-call register	No	It holds intermediate values between a procedure and the sub-procedure it calls.
r13 (SP)	Stack pointer	Yes	SP must be the same after a subroutine has completed.
r14 (LR)	Link register	No	LR does not have to contain the same value after a subroutine has completed.
r15 (PC)	Program counter	N/A	Do not directly change PC

Table 8-2. Standard of register usage of a subroutine

When using the push instruction to push multiple registers onto the stack, as shown in Example 8-3, the order in which the registers are listed in the bracket does not matter. These the registers are sorted in the descending order based on their numbers. The processor pushes the largest register first during the push, and pops the smallest register first during the pop.

Push and pop multiple registers	Equivalent assembly code
<pre> MOV r4, #4 MOV r5, #5 MOV r6, #6 PUSH {r4, r5, r6} ; The order in the bracket does not matter. ; The register list is sorted based on ; their names and the largest numbered ; register is pushed first. POP {r6, r4, r5} ; The order in the bracket does not matter. ; The smallest numbered register is popped ; off the stack first. ; result: r4=4, r5=5, r6=6 </pre>	<pre> MOV r4, #4 MOV r5, #5 MOV r6, #6 ; Equivalent code PUSH {r6} PUSH {r5} PUSH {r4} ; Equivalent code POP {r4} POP {r5} POP {r6} ; result: r4=4, r5=5, r6=6 </pre>

Example 8-3. Push and pop operations of the stack

8.5 Passing Arguments to Subroutine via Registers

An assembly subroutine may be called by standard C programs or by assembly programs developed independently by different programmers. Therefore, all subroutines must follow the ARM EABI protocol in passing arguments and in returning a result.

The protocol of passing arguments is summarized below.

- When the caller passes up to four 32-bit arguments to a subroutine, the caller places these arguments in registers `r0`, `r1`, `r2`, and `r3`.
- When a subroutine takes two 64-bit arguments, such as “long long” and “double” in C, the caller puts the first 64-bit argument in registers `r0` and `r1`, and the second 64-bit argument in registers `r2` and `r3`. The most significant 32 bits of these two arguments are in register `r1` and `r3`, respectively.
- When the caller passes a 128-bit argument, the argument is contained in registers `r0`, `r1`, `r2`, and `r3`.
- When there are more than four 32-bit arguments, the first four arguments come in via registers `r0` – `r3`, respectively, and the subroutine receives the rest arguments via the stack.

The protocol of returning a result is as follows.

- When a subroutine returns a 32-bit value, the return value is stored in register `r0`.
- If the result has 64 bits, it is stored in register `r0` and `r1`.
- If the return has 128 bits, it is stored in registers `r0`, `r1`, `r2`, and `r3`.

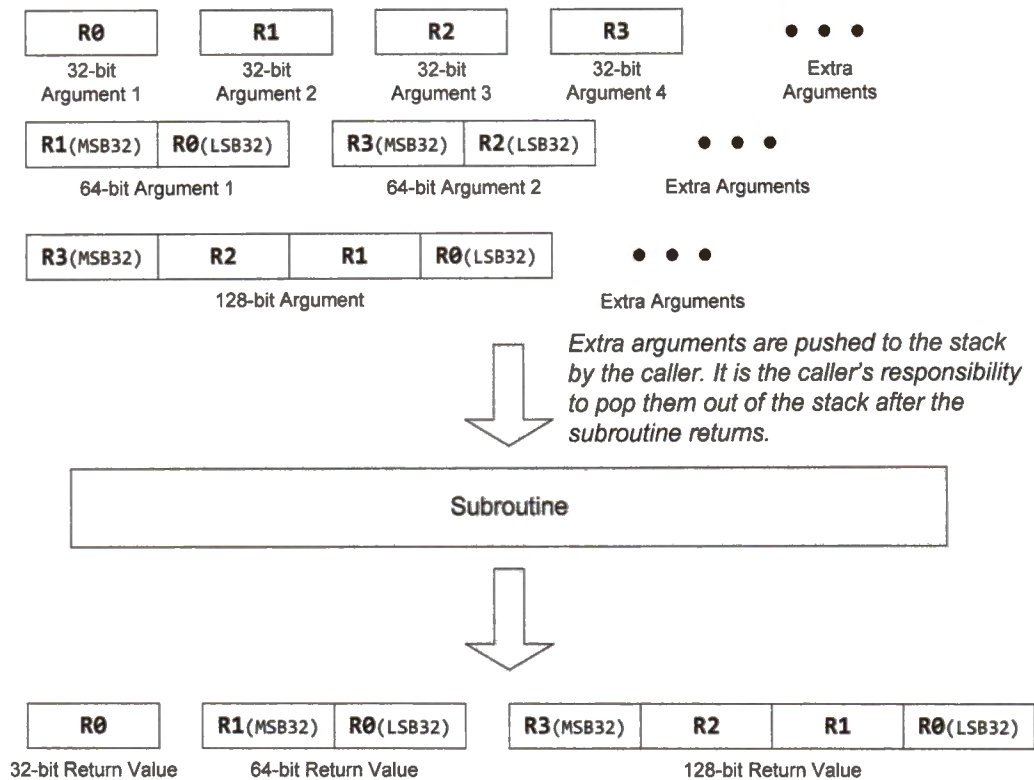


Figure 8-5. Passing arguments and returning a value. If a value takes multiple registers, the most significant 32 bits (MSB32) are stored in the highest numbered register.

8.5.1 Pass a Variable by Value and by Reference

Variables are stored in memory typically. A caller can pass variable arguments to a subroutine by value or by reference. In C, the arguments are always passed by value. However, C can use memory points to emulate passing a variable by reference. Figure 8-6 shows the comparison. If passed by value, the input argument remains unchanged.

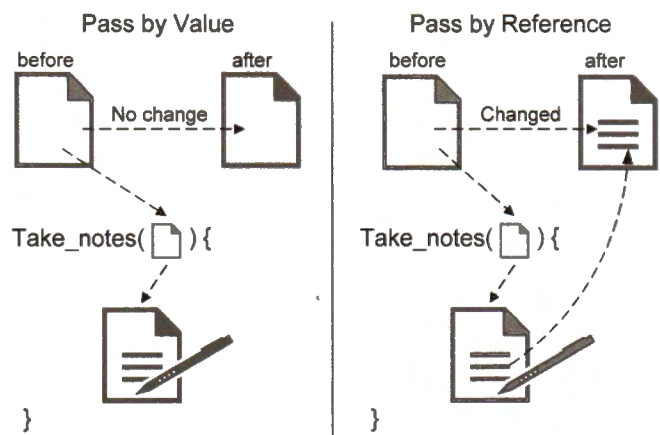


Figure 8-6. Comparison between pass by value and pass by reference.

- Passing a variable *by value* is to make a copy of the variable. After the callee returns, the caller's variable always remains unchanged.
- Passing a variable *by reference* is achieved by making a copy of the memory address of the caller's variable. The callee can change the value of the caller's variable because the callee knows the memory address of the caller's variable.

Example 8-4 shows a simple example that compares passing a variable to a subroutine by value and by reference. We can better understand their differences by looking at the assembly programs into which a C compiler translates them, as illustrated in Figure 8-7 and Figure 8-8.

Pass by Value	Pass by Reference
<pre>void fun(int n){ n = 1; // The value of n is passed // won't update caller's n } int main(void){ int n = 0; fun(n); // pass value of n printf("%d", n); return 0; }</pre>	<pre>void fun(int *n){ *n = 1; // The address of n // is passed. // dereference and // update memory } int main(void){ int n = 0; int *p = &n; fun(p); // pass pointer printf("%d", n); return 0; }</pre>
Program Output: 0	Program Output: 1

Example 8-4. Comparison of passing a variable by value and by reference in C

When compiling a C program, the compiler can assign a local variable to a register or store it in the heap region of the data memory, as discussed in Chapter 7.2. Suppose variable *n* in Example 8-4 is stored in the data memory. Figure 8-7 and Figure 8-8 compare the operations of passing a variable by value and by reference.

1. When a variable is *passed by value*, Figure 8-7 illustrates the key operations of the corresponding assembly code of Example 8-4.
 - The value of variable *n* is loaded from the memory to register *r0*.
 - While the subroutine changes the value of register *r0*, the value stored in memory remains unchanged.
 - Therefore, a subroutine cannot modify the in-memory variable if the variable is passed to the subroutine by value.
2. When a variable is *passed by reference*, Figure 8-8 shows the key operations of its assembly implementation of Example 8-4.

- The memory address of variable *n* is loaded into register *r0* and is passed to the subroutine.
- Register *r0* serves as a pointer to variable *n*.
- Since the memory address is sent to the subroutine, the subroutine can change the value of the variable by using the store instruction (STR).
- When the subroutine exits, the value of the caller's variable *n* in memory has been changed.

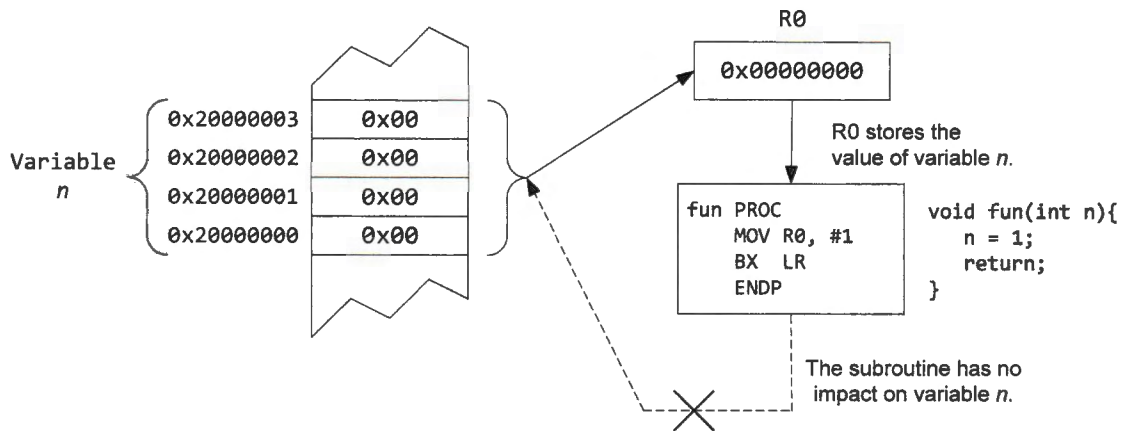


Figure 8-7. Example of passing a variable *by value*.

If variable *n* is passed to a subroutine by value, the subroutine cannot change the variable value stored in memory because the subroutine only knows the value but not the memory address of variable *n*. Therefore, after the *fun* subroutine exits, the value of *n* stored in memory is still 0, not 1.

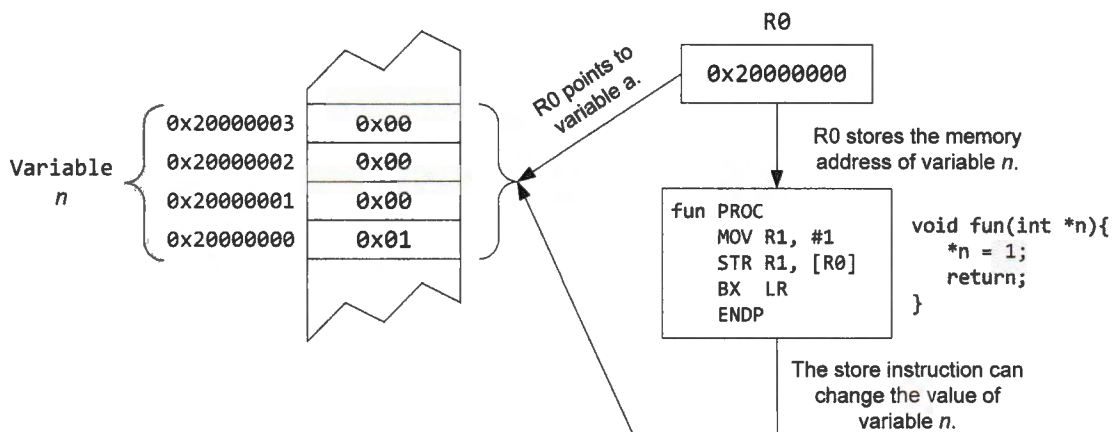


Figure 8-8. Example of passing a variable *by reference*.

If variable *n* is passed to a subroutine by reference, the memory address of variable *n* is passed. Therefore, the subroutine can change the value of variable *n*. After the subroutine exits, the value of *n* in memory is changed to 1.

8.5.2 Example of Passing by Value

In Example 8-5, two 32-bit integers are passed to the subroutine *sum2*. Per the application binary interface (ABI) protocol, the assembly program should meet the following requirements.

- The value of variable *a1* is received in register *r0*, and *a2* in register *r1*. An integer in C has 32 bits. Therefore, only one register is required to hold *a1*.
- The caller program assumes the subroutine saves the result *total* in register *r0*.
- Registers *r0*, *r1*, *r2*, and *r3* are scratch registers, and the subroutine does not have to preserve them in the stack. If the caller wishes to safeguard the value of these registers, the caller must push them onto the stack before the subroutine is called, and pop them off stack after the subroutine completes.
- If a subroutine does not call any other subroutine, the subroutine is required to preserve the link register (LR) in the stack. Otherwise, the subroutine should preserve LR on entry and recover it on exit.

C Program	Assembly Program
<pre>int t; int sum2(int a1, int a2); int main(){ t = sum2(1, 2); while(1); } int sum2(int a1, int a2){ int total; total = a1 + a2; return total; }</pre>	<pre>AREA sum, CODE EXPORT __main ALIGN ENTRY __main PROC MOV r0, #1 ; 1st argument MOV r1, #2 ; 2nd argument BL sum2 ; result returned in r0 ; LR points to LDR instr LDR r1, =t ; memory address of t STR r0, [r1] ; save the sum stop B stop ENDP sum2 PROC ; name of procedure ; r0 = 1st argument ; r1 = 2nd argument ADD r0, r0, r1 ; r0 = r0 + r1 ; r0 = result to be returned BX lr ; set PC to LR ENDP ; end of procedure AREA myData, DATA t DCD 0 END ; end of program</pre>

Example 8-5. Passing two arguments to a subroutine

In the following, we illustrate how to pass a 64-bit value to a subroutine. In C, a long long variable has 64 bits in memory. When a subroutine takes a long long variable, two registers, r0 and r1, are used to hold the value of this variable. Register r1 holds the most significant 32 bits, and register r0 contains the least significant 32 bits. For example, if the long long variable has a value of -2, then r0 is 0xFFFFFFFFE and r1 is 0xFFFFFFFF.

The subroutine returns the 64-bit result in register r1 and r0. In the following example, the 64-bit sum should be 3. Thus, when the subroutine returns, r1 is 0x00000000 (the upper word), and r0 is 0x00000003 (the lower word). If the 64-bit result should be saved in the data memory, two STR instructions are used to store the upper and lower words, respectively.

C Program	Assembly Program
<pre> long long t; long sum2(long long a1, long long a2); int main(){ t = sum2(1, 2); while(1); } long long sum2(long long a1, long long a2){ long long total; total = a1 + a2; return total; } </pre>	<pre> AREA sum, CODE EXPORT __main ALIGN ENTRY __main PROC MOV r0, #1 ; 1st 64-bit argument MOV r1, #0 ; r1:r0 MOV r2, #2 ; 2nd 64-bit argument MOV r3, #0 ; r3:r2 BL sum2 ; result in r1:r0 ; LR points to LDR instr. LDR r3, =t ; memory address of t STR r0, [r3] ; save lower 32 bits STR r1, [r3, #4] ; save upper 32 bits ; r0 = 3, r1 = 0 in ; this example stop B stop ENDP sum2 PROC ; name of procedure ; r1:r0 = 1st 64-bit argument ; r3:r2 = 2nd 64-bit argument ADDS r0, r0, r2 ; add lower 32 bits ADC r1, r1, r3 ; add upper 32 bits ; r1:r0 = 64-bit return value BX lr ; set PC to LR ENDP ; end of procedure AREA myData, DATA t DCQ 0 ; allocate 8 bytes END ; end of program </pre>

Example 8-6. Passing two 64-bit arguments to a subroutine

8.5.3 Write a Subroutine in Different Files

Subroutines can be in separate source files. A compiler must compile these source files and then link them together to build an executable. Placing subroutines in multiple files improves the clarity and makes the code more manageable when the source code is large.

A subroutine should export its name if it is called by codes in other source files. Exporting a subroutine allows other source files to reference it during the linking state. Similarly, a caller should import the subroutine name defined in a different source file.

Example 8-7 shows how an assembly program calls a subroutine implemented in a separate assembly source file.

- In the *sum2.s* file, “EXPORT sum2” is used to declare a symbol *sum2* that may be referred to in other source files. The directive “EXPORT” in assembly is like “EXTERN” in C.
- In the *main.s* file, “IMPORT sum2” is used to tell the assembler that the symbol *sum2* is defined in a different file.

Source file <i>sum2.s</i>	Source file <i>main.s</i>
<pre> AREA sum, CODE EXPORT sum2 ALIGN sum2 PROC ADD r0, r0, r1 ; return r0 BX lr ; set PC to LR ENDP END </pre>	<pre> AREA program, CODE EXPORT __main IMPORT sum2 ALIGN ENTRY __main PROC MOV r0, #1 ; 1st argument MOV r1, #2 ; 2nd argument BL sum2 ; return r0 stop B stop ENDP END </pre>

Example 8-7. Implementing a subroutine stored in a separate file. The keywords IMPORT and EXPORT are used to call a subroutine stored in a separate source file.

A symbol can be the name of a subroutine or a data variable. A compiler resolves all symbols at the linking stage in which various pieces of code and data are combined into a single executable file. A linker performs the linking process.

- The EXPORT directive makes a symbol visible to all modular files during the linking stage. By default, the linker tries to locate a symbol locally within the current source file or included files.
- The IMPORT directive informs the linker that a symbol is defined or implemented in a different file.

8.5.4 Example of Passing by Reference

Pass-by-reference in C is a method that passes the memory address of a variable to the subroutine and thus allows the subroutine to change the variable value. A C program must declare an argument passed by reference as a pointer type. The following *swap* subroutine takes two arguments that are passed by reference.

C program	Assembly Program
<pre>// swap two characters void swap (char *x, char *y) { char t; t = *x; *x = *y; *y = t; }</pre>	<pre>; swap routine swap PROC LDRB r2, [r0] LDRB r3, [r1] STRB r3, [r0] ; [r1] into [r0] STRB r2, [r1] ; [r0] into [r1] BX lr ENDP</pre>

Example 8-8. Implementation of passing by reference in assembly language

Example 8-9 shows a subroutine that swaps the first and the last character of a string. For example, the swap subroutine changes “abcde” to “ebcda”.

```
char str[6] = "abcde";    // Include the NULL terminator into the length
int main(void){
    swap(str, str+4);    // The result is "ebcda".
}
```

Example 8-9. A C program calls a subroutine implemented in assembly language.

In contrast to passing by reference, another method is to *pass by value*. The key difference is whether the caller can see any changes made to an argument by a subroutine.

The arguments in Example 8-10 are passed by value. The callee receives a copy of the variables’ values. These variables remain unchanged in the caller. Consequently, Example 8-10 fails to swap the characters in this string. The subroutine “swap(str[0], str[4])” does not affect the content of the string str at all.

C program (Incorrect code)	Assembly Program (Incorrect code)
<pre>// failed to swap x and y void swap (char x, char y) { char t; t = x; x = y; y = t; }</pre>	<pre>; swap routine swap PROC MOVS r2, r0 MOVS r0, r1 MOVS r1, r2 BX lr ENDP</pre>

Example 8-10. A subroutine implemented in C and assembly language fails to swap two characters when arguments are passed by value.

8.5.5 Example of Greatest Common Divisor

This example gives a subroutine that calculates the greatest common divisor (GCD) of two positive integers. Figure 8-9 shows the flowchart of the *gcd* subroutine.

The subroutine uses Euclid's algorithm, *i.e.*, $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$. The subroutine swaps a and b if a is smaller than b . It then repeatedly finds the GCD of the remainder ($a \bmod b$) and b . The remainder becomes smaller and smaller, and this process continues until the remainder becomes zero.

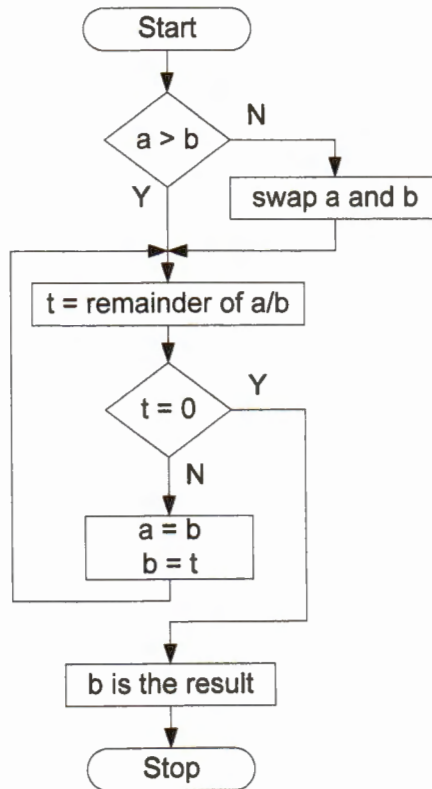


Figure 8-9. Flowchart of GCD subroutine

For illustration, suppose $a = 2310$ and $b = 483$. The program takes the following iterations:

1. $a = 2310$ and $b = 483$. Since $2310 = 4 \times 483 + 378$, the remainder of a/b is 378.
2. $a = 483$ and $b = 378$. Since $483 = 1 \times 378 + 105$, the remainder of a/b is 105.
3. $a = 378$ and $b = 105$. Since $378 = 3 \times 105 + 63$, the remainder of a/b is 63.
4. $a = 105$ and $b = 63$. Since $105 = 1 \times 63 + 42$, the remainder of a/b is 42.
5. $a = 63$ and $b = 42$. Since $63 = 1 \times 42 + 21$, the remainder of a/b is 21.
6. $a = 42$ and $b = 21$. Since $42 = 2 \times 21$, the remainder of a/b is 0.
7. The program stops, and the GCD of 2310 and 483 is 21.

Per the ARM EABI protocol, the two arguments *a* and *b* are passed in register *r0* and *r1*, respectively. The subroutine returns the result in register *r0*.

Since this *gcd* subroutine does not call any other subroutines, it does not have to preserve the link register (LR) in the stack. Also, the subroutine only uses registers *r0*, *r1*, *r2*, and *r3*. Thus, there is no need to push any registers onto the stack.

C Program	Assembly Program
<pre> int gcd(int a, int b); int result; int main(void){ result = gcd(21, 28); while(1); } int gcd(int a, int b) { int t; // temp variable // swap a and b if a < b if(a < b) { t = a; a = b; b = t; } while(b != 0){ t = a % b; a = b; b = t; } return a; } </pre>	<pre> AREA myData, DATA ALIGN result DCW 0 ; allocate four bytes AREA GCD, CODE EXPORT __main ALIGN ENTRY __main PROC MOV r1, #21 ; 1st argument MOV r0, #28 ; 2nd argument BL gcd ; call subroutine ; GCD is returned in r0 LDR r2, =result ; r2 = memory address STR r0, [r2] ; save result stop B stop ENDP gcd PROC ; r0 = 1st argument = a ; r1 = 2nd argument = b CMP r0, r1 ; compare a & b MOVLT r2, r0 ; if a < b, swap a & b MOVLT r0, r1 MOVLT r1, r2 loop CBZ r1, exit ; if b = 0, exit SDIV r3, r0, r1 ; r3 = r0/r1 MLS r2, r1, r3, r0 ; r2 = r1 - r3*r0 MOV r0, r1 ; a = b MOV r1, r2 ; b = remainder B loop ; loop again exit BX lr ; return in r0 ENDP END </pre>

Example 8-11. Finding the greatest common divisor (GCD)

8.5.6 Example of Concatenating Two Strings

When a string is concatenated to another string, the destination string must have enough extra memory space to hold the resulting concatenated string. Otherwise, data stored after the target string in memory may be modified by mistake. Thus, in the data region, the statement “`str1_ SPACE 20`” reserves 20 bytes of memory space.

Each string ends with a NULL character. This example uses the post index memory addressing. The following instruction with post-index addressing

```
LDRB r3, [r1], #1    ; post-index addressing
```

is equivalent to:

```
LDRB r3, [r1]
ADD  r1, r1, #1
```

C Program	Assembly Program
<pre>void strcat(char *s1, char *s2); char s1[20] = "Shaking"; char s2[10] = " hands"; int main(){ strcat(s1, s2); while(1); } // Concatenate two strings void strcat(char *dst, char *src){ while(*dst != '\0') dst++; while((*dst++ = *src++) != '\0'); return; }</pre>	<pre>AREA myData, DATA ALIGN str1 DCB "Shaking",0 str1_ SPACE 20 ; reserve space str2 DCB " hands",0 AREA my_strcat, CODE EXPORT __main ALIGN ENTRY __main PROC LDR r0, =str1 ; 1st argument LDR r1, =str2 ; 2nd argument BL strcat ; call subroutine stop B stop ENDP ; Concatenate two strings strcat PROC loop LDRB r2, [r0] ; Load a byte CBZ r2, copy ; null ending ADD r0, r0, #1 B loop copy LDRB r3, [r1], #1 ; post-index STRB r3, [r0], #1 ; post-index CBNZ r3, copy BX lr ENDP END</pre>

Example 8-12. Concatenating two strings in C and assembly

8.5.7 Example of Comparing Two Strings

The *strcmp* compares two null-terminated strings and returns a positive, zero, or a negative integer if the first string is greater than, equal to, or less than the second string, respectively.

The comparison starts with the first pair and continues with the next pair if the first pair is equal. When a pair is different from each other, the function *strcmp* returns the difference between this pair. The following gives a few examples.

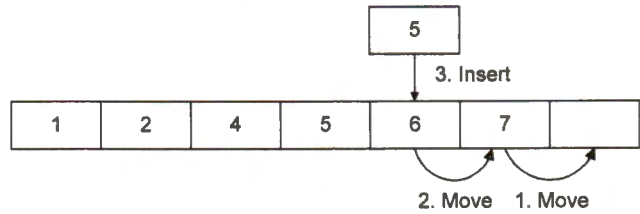
- *strcmp*("their", "there") returns -9 because ASCII of "i" and "r" is 105 and 114, respectively.
- *strcmp*("their", "the") returns 105. A string must end with the NULL character. The ASCII value of "i" is 105 and the NULL terminator is 0.
- *strcmp*("the", "there") returns -114. ASCII of NULL and 'r' is 0 and 114.
- *strcmp*("their", "their") returns 0.

C Program	Assembly Program
<pre> char str1[] = "dog"; char str2[] = "cat"; int result; int main(void){ int result; result = strcmp(str1, str2); while(1); } // Compare two strings int strcmp(char *s, char *t){ while(*s == *t){ if (*s == '\0') return 0; s++; t++; } return *s - *t; } </pre>	<pre> AREA myData, DATA str1 DCB "dog",0 ; NULL terminated str2 DCB "cat",0 ; NULL terminated result DCW 0 ; allocate one word AREA my_strcmp, CODE EXPORT __main ALIGN ENTRY __main PROC LDR r0, =str1 ; address of str1 LDR r1, =str2 ; address of str2 BL strcmp ; call subroutine LDR r1, =result ; address of result STR r0, [r1] ; save the result stop B stop ; dead loop ENDP ; Compare two strings strcmp PROC ; r0 = s, r1 = t loop LDRB r2, [r0], #1 ; post-index LDRB r3, [r1], #1 ; post-index CBZ r2, exit ; NULL terminator CMP r2, r3 ; if *s == *t BEQ loop ; Compare again exit SUB r0, r2, r3 ; r0 = *s - *t BX lr ; return r0 ENDP END </pre>

Example 8-13. Comparing two strings in C and assembly

8.5.8 Example of Inserting an Integer into a Sorted Array

The subroutine has three arguments, passed in registers *r0*, *r1*, and *r2*. The subroutine preserves registers *r4* and *LR*. In the end, it pops *LR* to *PC*, which makes it return to the caller.



Suppose the array has been sorted in ascending order. The program starts with the last element (*i.e.*, the largest one) of the array and compares the input integer with each array element one by one. If the element is greater than the input, this element is moved to the right by one; otherwise, the input is inserted at the current position.

C Program	Assembly Program
<pre>int a[10] = {1, 2, 4, 5, 6, 7}; void insert(int, int *, int); int main(void){ insert(3, a, 5); while(1); }</pre>	<pre>AREA myData, DATA ALIGN a DCD 1,2,4,5,6,7 AREA insert_array, CODE EXPORT __main ALIGN ENTRY __main PROC MOV r0, #3 ; 1st argument, value LDR r1, =a ; 2nd argument, array MOV r2, #5 ; 3rd argument, size BL insert ; call subroutine stop B stop ENDP ; r0 = value, r1 = array, r2 = size insert PROC PUSH {r4, lr} loop CMP r2, #0 ; check i > 0 BLE done ; done if i ≤ 0 SUB r4, r2, #1 ; r4 = i - 1 LDR r4, [r1,r4,LSL #2] ; a[i-1] CMP r0, r4 ; compare v & a[i-1] BGE done ; done if v ≥ a[i-1] STR r4, [r1,r2,LSL #2] ; a[i] SUB r2, r2, #1 ; i-- B loop done STR r0, [r1,r2,LSL #2] ; a[i] = v POP {r4, pc} ; exit ENDP END</pre>
<pre>// input: value, pointer, size void insert(int v, int *a, int s){ int i; for (i=s; i>0 && v<a[i-1]; i--){ a[i] = a[i-1]; } a[i] = v; return; }</pre>	

Example 8-14. Inserting an integer into a sorted array in C and assembly

8.5.9 Example of Converting Integer to String (itoa)

The C function *itoa* converts an integer to a string. To convert a digit d ($0 \leq d \leq 9$) to its ASCII value of the corresponding letter `ch[1]`, we can

$$\text{ch}[0] = d + 0x30$$

or

$$\text{ch}[0] = d + '0'$$

Letter	0	1	2	3	4	5	6	7	8	9
ASCII	0x30	0x31	0x32	0x33	0x34	0x35	0x36	0x37	0x38	0x39

For a given unsigned integer, such as 12345, we extract the digits backward. We start with the least significant digit, which can be obtained by using modulo operation to find the remainder of the division of 12345 by 10, *i.e.*, $\text{mod}(12345, 10)$. We append this digit to a string.

The modulo operation continues until we obtain all digits. In the end, we get a string of "54321". We need to reverse the string order and move the least significant digit from the first position to the end of the string.

Quotient/10	Quotient	Remainder	Reverse
12345/10 =	1234	5	↑↑
1234/10 =	123	4	
123/10 =	12	3	
12/10 =	1	2	
1/10 =	0	1	
	If quotient is 0, stop	Result = "54321"	Result = "12345"

12345/10 Quotient = 1234 Remainder = 5	1234/10 Quotient = 123 Remainder = 4	123/10 Quotient = 12 Remainder = 3	12/10 Quotient = 1 Remainder = 2	1/10 Quotient = 0 Remainder = 1
--	--	--	--	---

Figure 8-10. Basic steps of obtaining all digits of an integer

C Program	Assembly Program
<pre> char str[20]; void itoa(unsigned int, char *); int main(void){ itoa(12345, str); while(1); } void itoa(uint32_t n, char *s) { char * p = s, temp; // Build the string backward for (; n != 0; n /= 10){ *p = n % 10 + '0'; p++; } *p = '\0'; p--; // skip NULL // Reverse the string for(; p > s; s++, p--){ temp = *p; *p = *s; *s = temp; } return; } </pre>	<pre> AREA myData, DATA ALIGN str SPACE 20 AREA my_itoa, CODE EXPORT __main ALIGN ENTRY __main PROC MOV r0, #12345 LDR r1, =str BL itoa stop B stop ENDP itoa PROC PUSH {r4-r6, lr} MOV r2, r0 ; r2 = n MOV r3, r1 ; r3 = s ; Build the string backward MOV r6, #10 loop1 CBZ r2, done ; done if n = 0 UDIV r5, r2, r6 ; r5 = n/10 MLS r4, r6, r5, r2 ; r4 = r2-10*r5 ADD r4, r4, #0x30 ; n%10 + '0' STRB r4, [r3], #1 ; *p = n%10+'0' UDIV r2, r2, r6 ; n /= 10 B loop1 done MOV r4, #0 STRB r4, [r3] ; *p = '\0'; SUB r3, r3, #1 ; skip NULL ; Reverse the string loop2 CMP r3, r1 ; compare p & s BLE exit ; exit if p ≤ s LDRB r4, [r3] ; swap LDRB r5, [r1] STRB r4, [r1] STRB r5, [r3] ADD r1, r1, #1 ; s++ SUB r3, r3, #1 ; p-- B loop2 exit POP {r4-r6, pc} ENDP END </pre>

Example 8-15. Converting an integer to a string in C and assembly

8.5.10 Example of Matrix Transpose

In linear algebra, the transpose of a matrix $[a_{ij}]_{m \times n}$ is $[a_{ji}]_{n \times m}$. In C, a two-dimensional (2D) matrix in fact is stored as a one-dimensional array in memory. C uses a row-major approach to convert the 2D matrix into a 1D array, and it stores the matrix row by row contiguously in memory.

For example, a 3×3 matrix is stored as a simple 1D array in memory, as shown in Figure 8-11. We assume each element is an integer and takes four bytes. Because a memory address is always in terms of bytes, the memory offset is also expressed in bytes.

Index	(0, 0)	(0, 1)	(0, 2)	(1, 0)	(1, 1)	(1, 2)	(2, 0)	(2, 1)	(2, 2)
Content	1	2	3	4	5	6	7	8	9
Memory offset in bytes	0	4	8	12	16	20	24	28	32
	1 st Row			2 nd Row			3 rd Row		

Figure 8-11. Linear layout of a two-dimensional matrix in memory

We can replace multiplication with shift operations to improve the speed. For example, the following instructions

```
MOV r1, #3
MUL r4, r0, r1
```

can be replaced by a single instruction “ADD r4, r0, r0, LSL #1”, in which

$$r4 = r0 + r0 \ll 1 = 3 \times r0.$$

The shift and addition operations are faster than the multiplication instruction.

Figure 8-12 gives an example of matrix transpose. Note the items on the diagonal do not change. Therefore, the program only needs to swap elements in the upper right triangular matrix with the lower left triangle matrix.

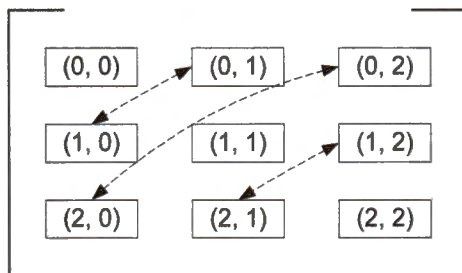


Figure 8-12. Demonstration of matrix transpose. None-diagonal elements are swapped.

C Program	Assembly Program
<pre> int a[3][3] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } }; void transpose(int *p); int main(void){ int *p = &a[0][0]; transpose(p); while(1); } void transpose(int *p){ int i, j, t; for(i=0; i<3; i++){ for (j=i+1; j<3; j++){ t = *(p + 3*i + j); *(p + 3*i + j) = *(p + 3*j + i); *(p + 3*j + i) = t; } } return; } </pre>	<pre> AREA myData, DATA ALIGN matrix DCD 1, 2, 3, 4, 5, 6, 7, 8, 9 AREA Matrix_Transpose, CODE EXPORT __main ALIGN ENTRY __main PROC LDR r0, =matrix BL transpose stop B stop ENDP transpose PROC PUSH {r4-r7, lr} MOV r1, #0 ; r1 = i; loop_i CMP r1, #3 BGE exit_i ADD r2, r1, #1 ; j = i + 1 loop_j CMP r2, #3 BGE exit_j ; r4 = p + (3*i + j)*4 ADD r4, r1, r1, LSL #1 ; 3 * i ADD r4, r0, r4, LSL #2 ; 4 * r4 ADD r4, r2, LSL #2 ; r5 = p + (3*j + i)*4 ADD r4, r2, r2, LSL #1 ; 3 * j ADD r4, r0, r4, LSL #2 ; 4 * r4 ADD r4, r1, LSL #2 ; swap elements pointed by r4 and r5 LDR r6, [r4] LDR r7, [r5] STR r7, [r4] STR r6, [r5] ADD r2, r2, #1 B loop_j ; for loop j exit_j ADD r1, r1, #1 ; i++ B loop_i ; for loop i exit_i POP {r4-r7, lr} ENDP END </pre>

Example 8-16. Transposing a matrix in C and assembly

8.5.11 Example of Removing a Character from a String

This example subroutine removes a specific character from a string. The subroutine starts to compare each character from the beginning and shift all following characters left one if this character is the same as the target characters. The subroutine adds a null terminator at the end of the string when it checks all characters.

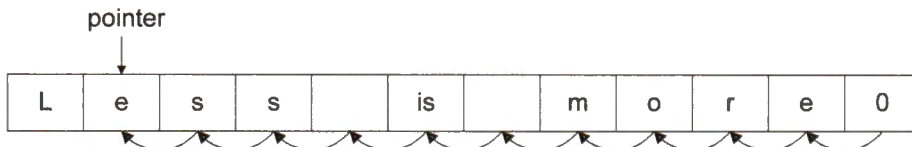


Figure 8-13. All following characters must be moved left when a character is removed.

C Program	Assembly Program
<pre>char s[15] = "Less is more"; void remove(char *s, char c); int main(void){ // remove Letter 'e' from s remove(s, 'e'); while(1); } // Remove c from string s void remove(char *s, char c){ char *t = s; for(; *s != '\0'; s++){ if (*s != c){ *t = *s; t++; } } *t = '\0'; return; }</pre>	<pre>AREA myData, DATA str DCB "Less is more",0 AREA removeChar, CODE EXPORT __main ALIGN ENTRY __main PROC LDR r0, =str ; memory address MOVS r1, #'e' ; ASCII of 'e' ; r0 and r1 are arguments BL remove stop B stop ; dead Loop ENDP remove PROC ; r0 = s, r1 = c MOV r2, r0 ; r2 = t = s loop LDRB r3, [r0] ; r3 = *s CBZ r3, exit ; null ending CMP r3, r1 ; compare *s & c LDRBNE r3, [r0] ; get byte *s STRBNE r3, [r2] ; store to *t ADDNE r2, r2, #1 ; t++ ADD r0, r0, #1 ; s++ B loop ; do it again exit STRB r3, [r2] ; *t = '\0'; BX lr ; return ENDP END</pre>

Example 8-17. Implementation of removing a character from a string

8.5.12 Example of Finding Unique Numbers in an Array

This example removes any duplicate numbers in an array. It finds all unique members of an array. As shown in Figure 8-14, the program uses three loops over the array.

- The outer loop based on loop variable i selects a character to be compared with.
- The middle loop based on j compares each of all following characters with $array[i]$.
- The inner loop based on loop variable k is used to shift all following characters to the left by one position if $array[j]$ is equal to $array[i]$.

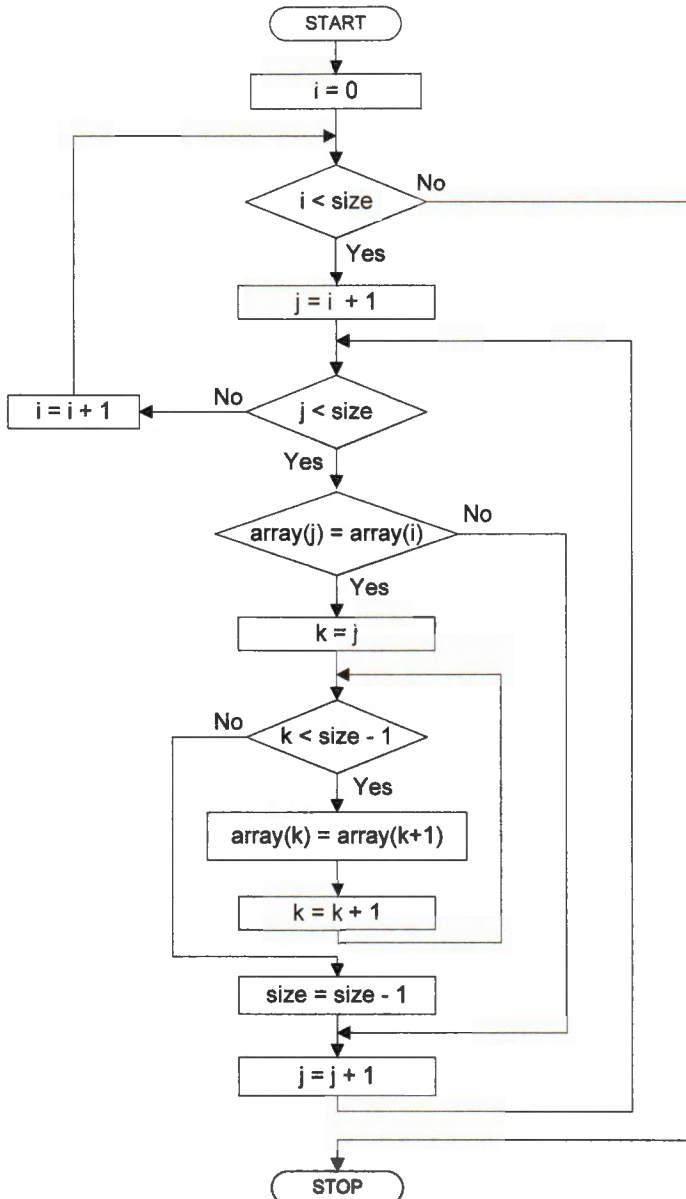


Figure 8-14. Flowchart of removing duplicates in an array

C Program	Assembly Program
<pre> int array[50] = { 7, 1, 7, 2, 1, 3, 1, 2, 4, 5, 2, 3, 2, 6, 7, 2, 3, 2, 6, 7}; int size = 20; int remove_dup(int *, int); int main(){ int i; size = remove_dup(array, size); for (i = size; i < 50; i++) array[i] = 0; while(1); } int remove_dup(int *array, int size) { int i, j, k; int *p; i = 0; while(i < size){ j = i + 1; while(j < size) { if(*(p+i)==*(p+j)){ for(k=j; k<size-1; k++) *(p+k) = *(p+k+1); size--; } else j++; } i++; } return size; } </pre>	<pre> AREA myData, DATA ALIGN array DCD 7, 1, 7, 2, 1 DCD 3, 1, 2, 4, 5 DCD 2, 3, 2, 6, 7 DCD 2, 3, 2, 6, 7 size DCD 20 AREA remove_duplicates, CODE EXPORT __main ALIGN ENTRY __main PROC LDR r0, =array LDR r1, =size LDR r1, [r1] BL remove_dup MOV r4, r0 ; r0 = size returned loop CMP r4, #50 BGE stop MOV r0, #0 STR r0, [r1,r4,LSL #2] ; array[i] ADD r4, r4, #1 B loop stop B stop ENDP remove_dup PROC ; r0 = array pointer ; r1 = size PUSH {r4-r8,lr} ; r5 = i, r6 = j, r7 = k MOV r5, #0 ; r5 = i Li CMP r5, r1 ; compare i and size BGE exit ADD r6, r5, #1 ; r6 = j, j = i + 1 Lj CMP r6, r1 ; compare j and size BGE Ej LDR r8, [r0,r5,LSL #2] ; r8 = *(p+i) LDR r4, [r0,r6,LSL #2] ; r4 = *(p+j) CMP r8, r4 ; *(p+i) and *(p+j) BNE Ek2 MOV r7, r6 ; r7 = k SUB r4, r1, #1 ; r4 = size - 1 Lk CMP r7, r4 ; compare k and size-1 </pre>


```

BGE Ek1
ADD r8, r0, r7, LSL #2
LDR r8, [r8, #4] ; r8 = *(p+k+1)
STR r8, [r0, r7, LSL #2] ; *(p+k)

ADD r7, r7, #1 ; k++
B Lk ; loop k

Ek1 SUB r1, r1, #1 ; size--
B Lj ; loop j

Ek2 ADD r6, r6, #1 ; j++
B Lj ; loop j

Ej ADD r5, r5, #1 ; i++
B Li ; loop i

exit MOV r0, r1 ; return size

POP {r4-r8, pc}

ENDP
END

```

Example 8-18. Removing duplicates in an integer array in C and assembly

Figure 8-15 shows an example to illustrate the basic idea of using three nested loops to eliminate all duplicates in an array.

- The first loop indexed by i starts with the first element.
- The second loop indexed by j iterates through all items between $(i+1)$ and the end of the array.
- When a duplicate is found, the third loop indexed by k is used to shift all numbers after the j^{th} number left by 1. In this example, when the program finds the duplicate "7", all numbers after the "7" is moved left by one.

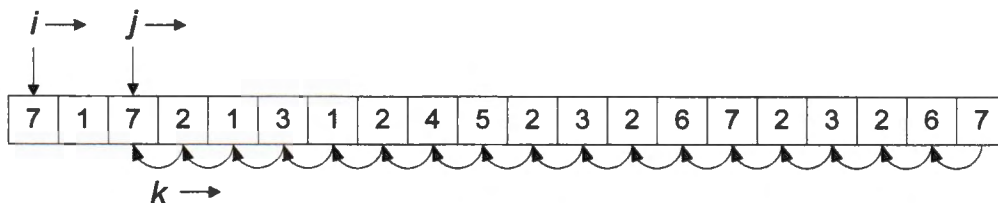


Figure 8-15. Variable i , j , and k are indices of three nested loops.
Loop k shifts numbers left after a duplicate is found.

8.6 Passing Arguments through Stack

A subroutine can receive up to four arguments via registers `r0-r3`. When a subroutine takes more than four arguments, additional parameters must be passed via the stack.

Example 8-19 shows a simple subroutine named *sum6* that takes six 32-bit integers and calculates their sum.

C Program	Assembly Program
<pre>int sum6(int a1, int a2, int a3, int a4, int a5, int a6); int main(){ int t; t = sum6(1, 2, 3, 4, 5, 6); while(1); } int sum6(int a1, int a2, int a3, int a4, int a5, int a6) { int total; total = a1 + a2 + a3 + a4 + a5 + a6; return total; }</pre>	<pre>AREA sum, CODE EXPORT __main ALIGN ENTRY __main PROC MOV r0, #5 ; 5th argument MOV r1, #6 ; 6th argument MOV r2, #3 ; 3rd argument MOV r3, #4 ; 4th argument PUSH {r0, r1} ; push 5th and 6th MOVS r1, #2 ; 1st argument MOVS r0, #1 ; 2nd argument BL sum6 stop B stop ENDP sum6 PROC PUSH {r4-r7, lr} MOV r4, r0 ; LDRD = Load double-word LDRD r5, r6, [sp, #20] ADD r7, r4, r1 ; add a1 & a4 ADD r7, r7, r2 ; add a2 ADD r7, r7, r3 ; add a3 ADD r7, r7, r5 ; add a5 ADD r0, r7, r6 ; add a6 POP {r4-r7, pc} ; return ENDP END</pre>

Example 8-19. Passing six arguments in C and assembly

The caller puts the first four arguments in registers `r0`, `r1`, `r2`, and `r3`, and pushes the last two arguments onto the stack. When the subroutine starts, it first should preserve any non-scratch registers used in this subroutine. In this example, the subroutine pushes registers `r4 - r7` and `LR` onto the stack.

The subroutine then uses a load double-word instruction (LDRD) to retrieve the additional two arguments by using SP-relative addressing.

```
LDRD r5, r6, [sp, #20]    ⇔    LDR r5, [sp, #20]
                                LDR r6, [sp, #24]
```

Note when a single push or pop instruction stores or loads multiple registers, the order in which these registers are listed in the instruction does not matter. The processor always pushes the largest register first, as introduced in Example 8-3. On the contrary, in a pop instruction, the value is popped to the register with the smallest register number first.

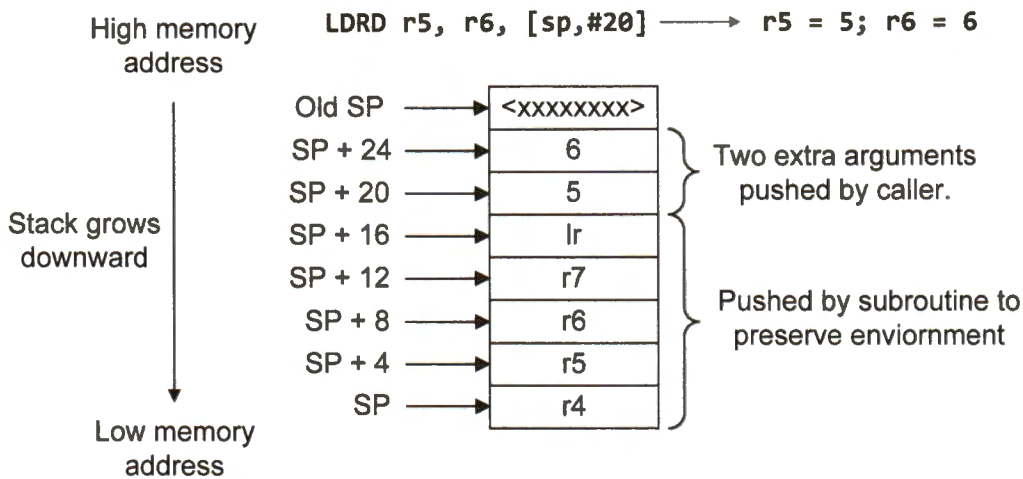


Figure 8-16. Memory layout after the subroutine preserves the environment

The LDRD instruction transfers two contiguous words starting at the memory address SP + 20 to two registers. The first destination register holds the word with a lower memory address. The LDRD instruction is equivalent to the following two LDR instructions:

```
LDR r5, [#sp, #20]    ; r5 = mem[sp + 20] = 5
LDR r6, [#sp, #24]    ; r6 = mem[sp + 24] = 6
```

The subroutine should not pop any arguments out from the stack. Per the application binary interface standard, the stack pointer (SP) must remain the same immediately before and after a subroutine is executed.

Example 8-20 gives a slightly more efficient implementation. The key idea is to reuse register r0 and, therefore, the subroutine does not need to push any registers onto the stack, reducing the number of memory accesses.

As well, because the subroutine *sum6()* does not call any other subroutines, it is not required to preserve the link register (LR) in the stack. Hence, it uses “BX LR” to return.

```

sum6 PROC
    ADD r0, r0, r1    ; add 1st and 2nd arguments
    ADD r0, r0, r2    ; add 3rd argument
    ADD r0, r0, r3    ; add 4th argument
    LDRD r2, r3, [sp] ; Load 5th and 6th arguments
    ADD r0, r0, r2    ; add 5th argument
    ADD r0, r0, r3    ; add 6th argument
    BX   LR          ; return
ENDP

```

Example 8-20. Improved implementation of the sum6 subroutine by reusing register r0

8.7 Recursive Functions

A *recursive function* is a function that calls itself directly or indirectly. If *foo()* calls *bar()*, and *bar()* calls *foo()*, then *foo()* calls itself indirectly. A recursive function solves a task by calling itself on smaller pieces of input data.

For example, *quick sort* is a recursive function. It randomly picks an element from an array and partitions the array into two subarrays, one with all elements smaller than the chosen element, and the other with all elements larger than the selected element. This process repeats on each subarray until there are only one or two items in each subarray. The result is a combination of these sorted subarrays.

A recursive function is an efficient divide-and-conquer tactic. It divides a large problem into smaller sub-problems of the same type, then solves those sub-problems, and finally combines the results to obtain the solution of the original problem.

Any problem solved by using a recursive function is also solvable by using a traditional iterative function based on loops. The advantages of recursive functions over iterative functions are twofold.

- A recursive function resembles the problem to be solved more naturally.
- Moreover, a recursive function is easier to program and debug in high-level languages such as C.

However, a recursive function usually is slower and requires more memory than its corresponding iterative function. Also, it is harder to debug a recursive function in assembly.

Example 8-21 shows recursive and iterative functions that calculate the factorial. Figure 8-7 shows the call graph of the recursive function when calculating *factorial*(5).

Recursive Function	Iterative Function
<pre> int factorial(int n) { if(n==1) return 1; else return n * factorial(n-1); } int main(void){ int y; y = factorial(5); return 0; } </pre>	<pre> int factorial(int n) { result = 1; for (int i = 1; i < n; i++) result *= i; return result; } int main(void){ int y; y = factorial(5); return 0; } </pre>

Example 8-21. Factorial function implemented by using recursive and iterative function

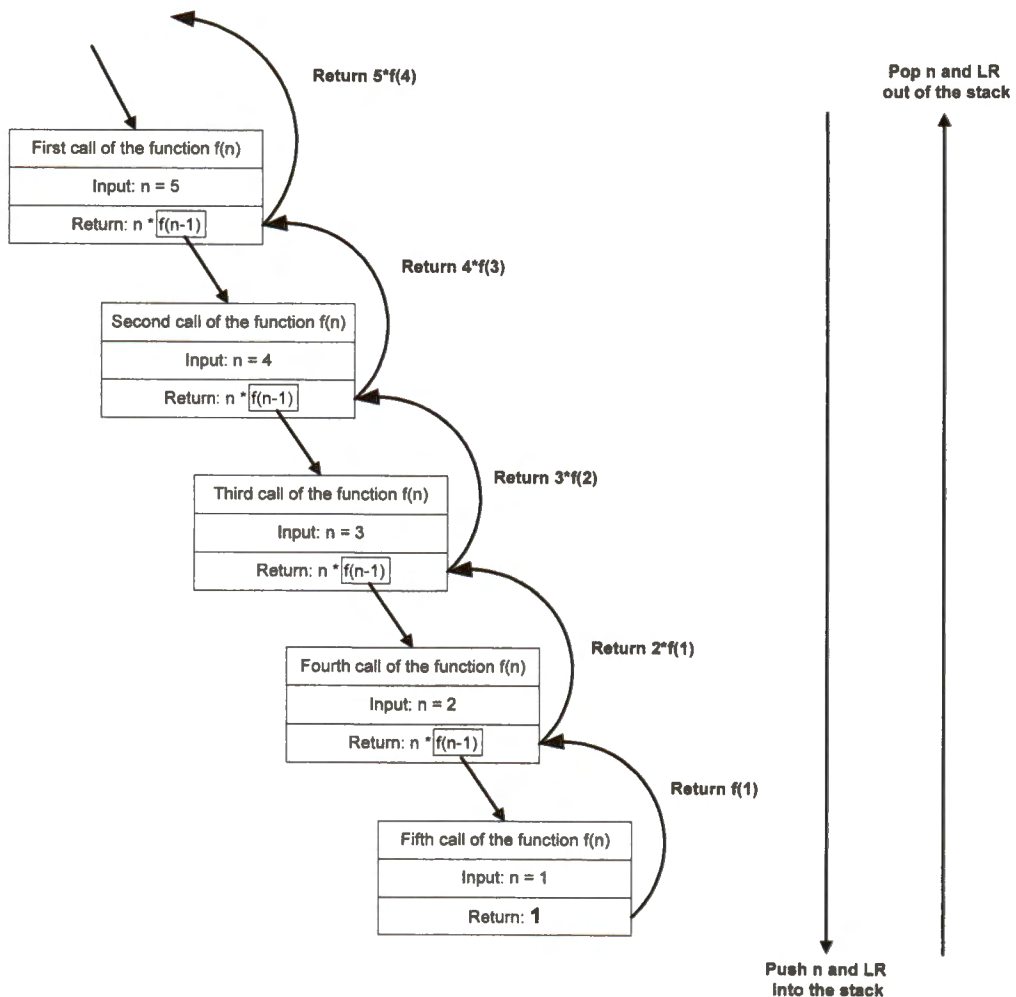


Figure 8-17. Call graph of the recursive factorial function

Each recursive function must have an *exit condition*, also known as a stopping case. The exit condition is to ensure that the recursive function does not go into an infinite loop. For example, the statement of “if (n == 1) return 1;” is the exit condition.

Figure 8-7 shows how the function *factorial*(5) runs step by step.

- *factorial*(5) calls *factorial*(4), and
- *factorial*(4) calls *factorial*(3), and
- *factorial*(2) calls *factorial*(1).
- After the exit condition is satisfied, *factorial*(1) returns the result backward to *factorial*(2) for calculating previously pending results.
- *factorial*(2) returns the result to *factorial*(3).
- *factorial*(3) returns the result to *factorial*(4).
- *factorial*(4) returns the result to *factorial*(5).
- *factorial*(5) completes and returns the final result.

Recursive functions utilize the stack to preserve the runtime environment and keep track of different call instances of the recursive functions. At least, a recursive function must preserve the link register (LR) in the stack. In the example of recursively calculating the factorial number, the subroutine must push LR and the input *n* onto the stack. The stack keeps growing as more data are pushed onto it during the recursive call. After the exit condition is satisfied, the stack begins to shrink as data are popped off the stack upon each function return.

In sum, when programming a recursive function in assembly, the function must manage the stack carefully to ensure the correctness and avoid infinite loop or stack overflow.

8.7.1 Example of Factorial Numbers

This section illustrates how the stack grows as a recursive function is called and how the stack shrinks when a recursive function is returned.

Suppose SP is 0x20000600, and r4 is 0 before the recursive factorial function starts. At the beginning of the subroutine, r4 and LR are pushed onto the stack. Recall that the BL instruction puts the memory address of the instruction immediately after the BL instruction into LR. The subroutine copies r0 to r4 because r0 is used to pass the argument when it calls itself.

Table 8-3 shows the stack content immediately after *factorial*(1) completes. The stack grows down toward the low memory address. Register r0 holds the input *n* passed to the subroutine. Because register r0 must hold the return result, the function copies r0 to r4. When the factorial function calls itself, LR points to the multiplication instruction “MUL r0, r4, r0”, which multiplies *n* and *factorial*(*n*-1).

Memory Address	Memory Content
0x20000600	
0x200005FC	0x08000134 (LR)
0x200005F8	0 (r4)
0x200005F4	0x08000148 (LR)
0x200005F0	5 (r4)
0x200005EC	0x08000148 (LR)
0x200005E8	4 (r4)
0x200005E4	0x08000148 (LR)
0x200005E0	3 (r4)
0x200005DC	0x08000148 (LR)
0x200005D8	2 (r4)
0x200005D4	0x08000148 (LR)
0x200005D0	

Table 8-3. Stack content immediately after factorial (1) completes.

C Program	Address	Assembly Program
<pre> int factorial(int n); int main(void){ factorial(5); while(1); } int factorial(int n) { int f; if(n==1) f = 1; else f = n*factorial(n-1); return f; } </pre>	<pre> 0x0800012E 0x08000130 0x08000134 0x08000136 0x08000138 0x0800013A 0x0800013C 0x0800013E 0x08000140 0x08000142 0x08000144 0x08000148 0x0800014C </pre>	<pre> AREA main, CODE, READONLY EXPORT __main ENTRY __main PROC MOV r0, #5 BL factorial stop B stop ENDP factorial PROC PUSH {r4, lr} ; preserve MOV r4, r0 ; r4 = n CMP r4, #1 BNE else ; if n ≠ 1 MOV r0, #1 ; f = 1 loop POP {r4, pc} ; return else SUB r0, r4, #1 ; n - 1 BL factorial ; r0 is input MUL r0, r4, r0 ; n*f(n-1) B loop ENDP END </pre>

Example 8-22. Calculating factorial number in C and assembly

8.7.2 Example of Reversing a String

The reverse of “ABCD” is “DCBA”. The following is a recursive reverse function. The key idea is to swap the first and last character, and reverse the substring excluding the first and the last characters. This process repeats for each substring.

C Program	Assembly Program
<pre> char str[20] = "Reverse me, please!"; void swap (char *x, char *y); void reverse(char *, int, int); int main() { reverse(str, 0, 20); while(1); } // swap two characters in a string void swap (char *x, char *y) { char temp; temp = *x; *x = *y; *y = temp; } // recursive function for reversion void reverse(char *str, int start, int end) { if (start == end) return; swap (str + start, str + end); start++; end--; reverse(str, start, end); } </pre>	<pre> AREA myData, DATA ALIGN str DCB "Reverse me, please!",0 AREA reverse_string, CODE EXPORT __main ALIGN ENTRY __main PROC LDR r0, =str ; 1st argument MOV r1, #0 ; 2nd argument MOV r2, #20 ; 3rd argument BL reverse ; Recursive call stop B stop ENDP swap PROC ; Swap routine LDRB r2, [r0] ; temp = *x LDRB r3, [r1] ; STRB r3, [r0] ; *x = y STRB r2, [r1] ; *y = temp BX lr ENDP reverse PROC PUSH {r4-r6, lr} MOV r6, r0 ; string pointer MOV r4, r1 ; start position MOV r5, r2 ; end position CMP r4, r5 ; check start <= end exit POPEQ {r4-r6, pc} ; exit ADD r0, r6, r4 ; [str + start] ADD r1, r6, r5 ; [str + end] BL swap ADD r4, r4, #1 ; start++ SUB r5, r5, #1 ; end-- MOV r0, r6 ; 1st argument MOV r1, r4 ; 2nd argument MOV r2, r5 ; 3rd argument BL reverse POP {r4-r6, pc} ENDP END </pre>

Example 8-23. Reversing a string in C and assembly

8.7.3 Example of String Permutation

The following code finds all possible permutations of the characters in a string. For example, the permutation of "ABC" includes: "ABC", "ACB", "BAC", "BCA", "CAB", and "CBA". The program stores all permutations in a string named *result*, separated by space. The *permute* function uses two subroutines: *strcat* that concatenates two strings (given in Chapter 8.5.6) and *swap* that swaps two characters in a string (given in Chapter 8.7.2). As shown in Figure 8-18, the permutation of a string is found by recursively permuting all new substrings. New substrings are formed by swapping the first letter with all letters in the original string (excluding the first letter of each new string).

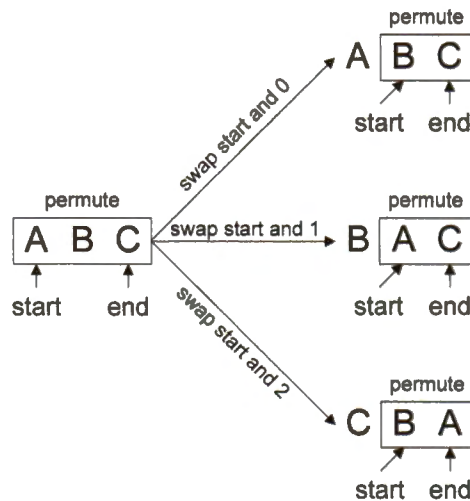


Figure 8-18. Permutation is achieved by swapping the first letter to every letter and permuting all substrings excluding the first letter of all new strings.

C Program	Assembly Program
<pre>char str[4] = "ABC"; char result[200]; char sep[2] = " "; void strcat(char *, char *); void swap(char *, char *); void permute(char *, int, int); int main() { permute(str, 0, 2); while(1); }</pre>	<pre>AREA myData, DATA ALIGN str DCB "ABC",0 result SPACE 200 sep DCB " ",0 AREA permute, CODE EXPORT __main ALIGN ENTRY __main PROC LDR r0, =str MOV r1, #0 ; start MOV r2, #2 ; end BL permute stop B stop ENDP</pre>

```
// recursive permute function
void permute(char *str,
             int start,
             int end) {

    int i;

    if (start >= end){

        strcat(result, sep);

        strcat(result, str);

    } else {

        for (i=start; i<=end; i++){

            swap(str + start, str + i);

            permute(str, start+1, end);

            swap(str + start, str + i);

        }
    }
    return;
}
```

```
; recursive permute function
permute PROC

    PUSH {r4-r7, lr}

    MOV r4, r0      ; r0 = *str
    MOV r5, r1      ; r1 = start
    MOV r7, r2      ; r2 = end

    CMP r5, r7      ; start >= end
    BLT else        ; if less or equal

then LDR r0, =result
    LDR r1, =sep
    BL  strcat      ; result + set

    LDR r0, =result
    MOV r1, r4      ; r1 = str
    BL  strcat      ; result + str

    B   exit

else MOV r6, r5      ; r6 = variable i

loop ADD r0, r4, r5   ; r0 = str + start
    ADD r1, r4, r6   ; r1 = str + i
    BL  swap

    MOV r0, r4      ; str
    ADD r1, r5, #1   ; start + 1
    MOV r2, r7      ; end
    BL  permute      ; recursive call

    ADD r0, r4, r5   ; str + start
    ADD r1, r4, r6   ; str + i
    BL  swap

    ADD r6, r6, #1   ; i++
check CMP r6, r7     ; compare i & end
    BLE loop

exit POP {r4-r7, pc}

    ENDP

    END
```

Example 8-24. String permutation in C and assembly

8.8 Exercises

1. Which of the following is equivalent to “PUSH {r7}”?
 - (1) $SP = SP - 4$, and then $memory[SP] = r7$
 - (2) $SP = SP + 4$, and then $memory[SP] = r7$
 - (3) $memory[SP] = r7$, and then $SP = SP - 4$
 - (4) $memory[SP] = r7$, and then $SP = SP + 4$
2. Which of the following instructions load five words from the stack into registers r1-r5? What are the value of registers r1-r5? (Assuming $SP = 0x20008000$, and these instructions are executed separately.)

- (1) LDMDA sp, {r1-r5}
- (2) LDMDB sp, {r1-r5}
- (3) LDMIA sp, {r1-r5}
- (4) LDMIB sp, {r1-r5}
- (5) LDMDA sp!, {r1-r5}
- (6) LDMDB sp!, {r1-r5}
- (7) LDMIA sp!, {r1-r5}
- (8) LDMIB sp!, {r1-r5}

Memory Address	Value
0x20008008	0x00000006
0x20008004	0x00000005
0x20008000	0x00000004
0x2000800C	0x00000003
0x20008008	0x00000002
0x20008004	0x00000001
0x20008000	0x00000000
0x20007FFC	0xFFFFFFFF
0x20007FF8	0xFFFFFFFEE
0x20007FF4	0xFFFFFFFED
0x20007FF0	0xFFFFFFFEC
0x20007FE8	0xFFFFFFFEB
0x20007FE4	0xFFFFFFFEA

3. What are the memory content after each of the following instruction completes? Assume register r1 = i, i = 0, 1, 2, 3, and 5. (Assuming $SP = 0x20008000$ and these instructions runs independently).
 - (1) STMDA sp, {r1-r5}
 - (2) STMDB sp, {r1-r5}

```

(3) STMIA sp, {r1-r5}
(4) STMIB sp, {r1-r5}
(5) STMDA sp!, {r1-r5}
(6) STMDB sp!, {r1-r5}
(7) STMIA sp!, {r1-r5}
(8) STMIB sp!, {r1-r5}

```

Memory Address	a	b	c	d	a	b	c	d
0x20008008								
0x20008004								
0x20008000								
0x2000800C								
0x20008008								
0x20008004								
0x20008000								
0x20007FFC								
0x20007FF8								
0x20007FF4								
0x20007FF0								
0x20007FE8								
0x20007FE4								

4. How many bytes does the stack need to pass the arguments when each of the following function is called?

```

(1) int32_t fun1(uint8_t a, uint16_t b, uint8_t c, int32_t d)

(2) int32_t fun2(uint8_t a, uint16_t b, uint8_t c, int32_t d,
                uint8_t e, int32_t f, uint8_t g)

(3) int32_t fun3(uint8_t a, int32_t b, int64_t c,
                uint8_t d, int32_t e)

(4) int32_t fun4(uint8_t a, int64_t b, int64_t c,
                uint8_t d, int32_t e)

```

5. Which register(s) holds the return value in the following functions?

```

(1) int16_t fun1()
(2) int8_t * fun2()
(3) int32_t * fun3()
(4) int64_t fun4()
(5) int64_t * fun5()

```

6. Why does a recursive assembly function have to preserve the link register?
7. Give two different assembly instructions to make a subroutine return.

8. Write a subroutine that checks whether a given number is a prime number. The subroutine takes one argument and returns true or false. Find all prime numbers between 100 and 200.
9. Write a subroutine that takes eight integer arguments and computes the product of these integers. The caller passes extra arguments to the subroutine via the stack.
10. Implement a subroutine of the Caesar shift encryption. It is a simple substitution encryption algorithm, in which each letter is replaced by another letter with a fixed number of offset down in the alphabet. For example, with a shift offset of 3, A becomes D, and B is substituted by E, and so on.
11. Write a subroutine called *MoviePrice* that calculates the movie ticket price based on the input argument called *age*. If the age is 12 or under, the price is \$6. If the age is between 13 and 64, the price is \$8. If the age is 65 or over, the price is \$7.
12. Write a subroutine that calculates the value of the following expression based on two input arguments a and n .

$$S_n(a) = a + aa + aaa + \cdots + \overbrace{aa \dots a}^n$$

For example, when $a = 3$ and $n = 5$, we have

$$S_5(3) = 3 + 33 + 333 + 3333 + 33333$$

13. Write a program that calculates $\sum_{n=0}^{10} n!$. The program should use two subroutines. One subroutine calculates the factorial $n!$, and the other subroutine calculates the sum of the factorials.
14. Write a program that uses a subroutine to find how many 1-bits exist in a 32-bit number.
15. Write a program that uses a subroutine to determine how many bits differ in two 32-bit numbers.
16. Mathematically, the cardinality of an array is defined as the number of unique elements in an array. Write a subroutine that calculates the cardinality of an integer array.
17. Write an assembly subroutine named f that calculates the following value

$$f(x, y) = ax^2 + bxy + c$$

where a , b , and c are constant integers, x and y are the input integers. Assuming a , b , and c are defined in the data memory, x and y are input arguments of this subroutine.

18. Write a recursive assembly subroutine that calculates the Fibonacci number.

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{otherwise} \end{cases}$$

19. When PC is $0x08000100$ in the following assembly program, the stack pointer (SP) is $0x20002000$. Show the value of the link register (LR) and the whole stack content when PC = $0x08000120$.

Memory Address	Instruction
$0x08000100$	MOV R0, #2
$0x08000104$	BL QUAD
$0x08000108$	B ENDL
$0x0800010C$	SQ1 PUSH {LR}
$0x08000110$	MUL R0, R0
$0x08000114$	BL SQ2
$0x08000118$	POP {PC}
$0x0800011C$	SQ2 PUSH {LR}
$0x08000120$	MUL R0, R0
$0x08000124$	POP {PC}
$0x08000128$	QUAD PUSH {LR}
$0x0800012C$	BL SQ1
$0x08000130$	POP {LR}
$0x08000134$	BX LR
$0x08000138$	ENDL ...

20. Write a recursive assembly subroutine that checks whether a given string is a palindrome.
21. Write a recursive assembly subroutine that calculates the K^{th} power of 2 (i.e., 2^K).
22. Compared with iterative methods, what are the advantages and disadvantages of recursive methods?